

FIAP — Global Solution 2026/1

DevOps Tools & Cloud Computing

2º ano — Análise e Desenvolvimento de Sistemas

Tema: Economia Espacial

Solução: PM_API (Problems Monitoring) — monitoramento de datacenters por sensores

Equipe

RM	Nome completo	Turma
RM561857	Nicolas Mota Cândido (representante)	2TDSPI
RM562979	Caio Kenzo Tayra	2TDSPI
RM563000	Enzo Vieira Bernardini	2TDSPI

Links da entrega

Repositório GitHub: [\[PREENCHER COM A URL DO REPOSITORIO\]](#)

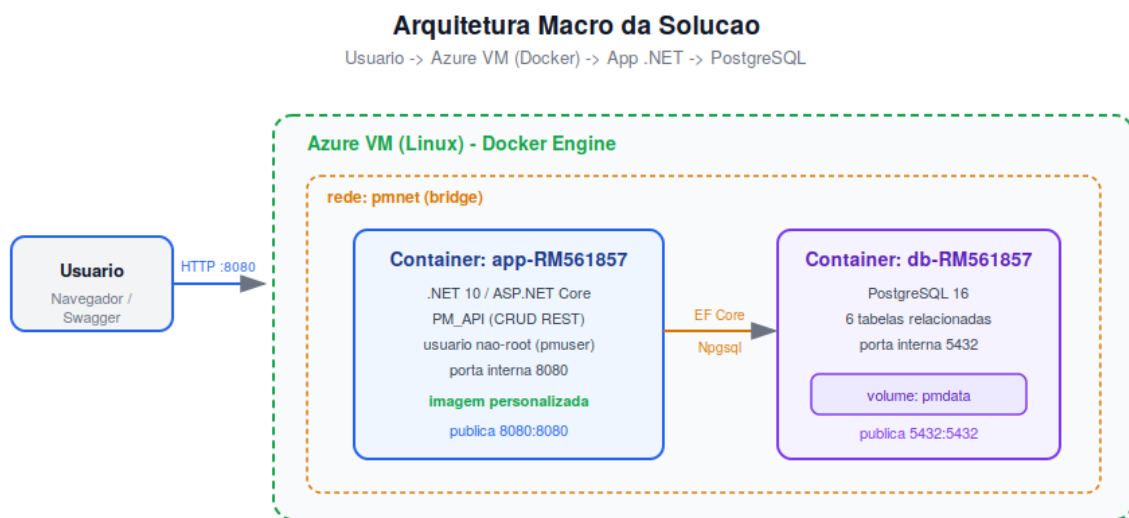
Vídeo demonstrativo (YouTube): [\[PREENCHER COM A URL DO VIDEO\]](#)

1. Descrição da solução

A solução PM (Problems Monitoring) é uma API REST para monitoramento de datacenters por meio de sensores, conectada ao tema da economia espacial (monitoramento de infraestrutura crítica de solo, de apoio a operações orbitais). A API expõe um CRUD completo sobre seis entidades relacionadas (DataCenter, Sensor, TipoAlerta, Alerta, Funcionário e Manutenção), persistindo os dados em um banco PostgreSQL.

Para a disciplina de DevOps, a aplicação foi containerizada e executada em ambiente de nuvem (Azure VM Linux) por meio de Docker Compose, com dois containers integrados na mesma rede: um container para a aplicação .NET (imagem personalizada gerada via Dockerfile, executando com usuário não privilegiado) e um container para o banco PostgreSQL (com volume nomeado para persistência dos dados).

2. Desenho macro da arquitetura



Fluxo: o usuário acessa a aplicação pela porta 8080 publicada na VM. O container do app (.NET) comunica-se com o container do banco (PostgreSQL) pela rede interna pmnet, usando o nome do serviço como host. Os dados persistem no volume nomeado pmdata.

3. Tecnologias utilizadas

.NET 10 / ASP.NET Core Web API · Entity Framework Core + Npgsql · PostgreSQL 16 · Docker e Docker Compose · Swagger / OpenAPI.

4. Requisitos atendidos

Requisito	Onde / como
Imagem personalizada do app	Dockerfile multi-stage
Usuário não privilegiado	Dockerfile (pmuser)
Diretório de trabalho definido	Dockerfile (WORKDIR /app)
Variável de ambiente no app	ConnectionStrings__DefaultConnection
Porta exposta do app	8080

Nome do container do app com RM	app-RM561857
CRUD completo em ≥ 2 tabelas	6 entidades relacionadas
App na mesma rede do banco	rede pmnet
Volume nomeado no banco	pmdata
Variável de ambiente no banco	POSTGRES_DB / USER / PASSWORD
Porta exposta do banco	5432
Nome do container do banco com RM	db-RM561857
≥ 2 tabelas com relacionamento	6 tabelas com chaves estrangeiras
Containers em background	docker compose up -d

5. Como executar (resumo)

O passo a passo completo, desde o `git clone` até as evidências de persistência com `SELECT` no banco, está documentado no arquivo `README.md` do repositório. Comandos principais:

```
git clone <repo> && cd <repo>
docker compose up -d --build
docker compose ps
docker compose logs app-RM561857
docker container exec -it app-RM561857 sh -c "pwd && ls -l && whoami"
docker container exec -it db-RM561857 psql -U pmuser -d pmdb -c "SELECT * FROM
datacenter;"
```